

aptana® Jaxer® The world's first Ajax server

Home Quick Start **Guide** Tutorials API Screencasts Forums Download

Jaxer Guide

-  [Introduction](#)
-  [Setup](#)
-  [Develop](#)
-  [Debug](#)

Folders

Folder Basics

Important Folder Locations

Jaxer.System provides several convenient properties for locating important system folders in a cross platform manner.

- **User folders:** The Desktop folder of the user account running Jaxer is located with the Jaxer.System.desktopFolder, and the home folder is located with Jaxer.System.homeFolder.
- **Jaxer Binaries:** The Folder containing the Jaxer binaries is located with the Jaxer.System.executableFolder
- **Jaxer Temporary Folder:** The Jaxer temporary folder is located with the Jaxer.System.tempFolder

```
1. var desktopFolder =
   ↵ Jaxer.System.desktopFolder;
2. var homeFolder = Jaxer.System.homeFolder;
3. var executableFolder =
   ↵ Jaxer.System.executableFolder;
4. var tempFolder = Jaxer.System.tempFolder;
```

Contents

[Folder Basics](#)
[Operating Systems and Paths](#)
[Simple Folder Access](#)
[Using Folder Objects](#)
[Navigating the filesystem](#)
[DirUtils and FileUtils](#)

API Links

[Jaxer.Dir](#)
[Jaxer.DirUtils](#)
[Jaxer.FileUtils](#)

Simple Folder vs. Folder Objects

Whether you create an object or use the static methods, Jaxer creates a Folder object, so the key reason to use a Folder object boils down to convenience. If you'll only be calling one or two methods against the folder then static is simpler; for more complex work create an object and store it in a variable for reuse.

Operating Systems and Paths

Working with Filepaths

Jaxer can handle filesystem paths as well as file:/// urls and provides convenience methods to resolve provided paths to an actual location on the filesystem.

Converting between a filesystem path and a URL

```
1. var fileUrl = Jaxer.Dir.pathToUrl("c:\\aptana")
2. // fileUrl -> file:///c:/aptana/
3. var filePath = Jaxer.Dir.urlToPath("file:///c:/aptana");
4. // filePath -> c:\aptana
```

Resolve a relative file path into an absolute path

```
1. var indexPath = Jaxer.Dir.resolve("index.html")
2. // indexPath -> C:\aptana\source\public\fileGrep\index.html
3. var folderPath = Jaxer.Dir.resolve(".")
4. // folderPath -> C:\aptana\source\public\fileGrep
5. var parentPath = Jaxer.Dir.resolve("..")
6. // parentPath -> C:\aptana\source\public
7. var grandParentPath = Jaxer.Dir.resolve("../..")
8. // grandParentPath -> C:\aptana\source
```

Using a combination of these methods will generally allow you to manipulate a provided path or Url to the required path.

Cross Platform Filepaths

Linux, Mac OS X and other *NIX operating systems require a forward slash to separate folder names when specifying the path for a file while MS Windows uses a back slash.

Further complicating things is that to include a backslash in a string in JavaScript you work around the fact that JavaScript uses backslash as a signal to escape the following character(s) and treat them as literals rather than commands—\n would normally be treated as the new line character. To escape a backslash you must use two ("\\"). One method Jaxer provides to deal easily with these issues, a good choice for when you need to build a string to keep for use later, is Jaxer.System.separator. It returns the system path separator, as a string.

```
1. var folderUrl = Jaxer.Dir.pathToUrl("c:" + Jaxer.System.separator + "aptana");
2. // folderUrl -> file:///c:/aptana/
```

A cleaner way to do this is with Jaxer.Dir.combine(args) method, which concatenates a list of parts into a full path joined with the platform path separator and cleans any extra path separators contained in the list.

```
1. var newPath = Jaxer.Dir.combine(Jaxer.System.tempFolder, 'afolder', 'afile.txt')
2. // on windows -> C:\aptana\JaxerDev\tmp\afolder\afile.txt
3. // on linux -> /opt/aptana/jaxer/afolder/afile.txt
```

Simple Folder Access

Checking whether a Folder Exists

Use `Jaxer.Dir.exists()`, which returns true if the path exists on the filesystem and false if not.

```
1. Jaxer.Dir.exists(Jaxer.System.tempFolder);
2. // returns true as tempFolder resolves to a real part of the filesystem
```



Finding Filenames which Match a Pattern

Use `Jaxer.Dir.grep()` to get a list of file objects that match a specified pattern:

```
1. var myResults = Jaxer.Dir.grep(Jaxer.Dir.resolve('.'), {pattern: '^.*html$'});
```



This example returns an array of all the html files contained in the current folder. [Navigating the filesystem](#) has a more in depth review of this functionality.

Using Folder Objects

Instantiating

Create a folder object by newing up `Jaxer.Dir(pathToFile)`. However, creating a folder object *does not* automatically create the equivalent folder on the file system. To make the folder real, you may either call `create()` on a dir object you previously instantiated or use the static version of `create()`, passing the resolved `pathToFile` as a parameter. Both `create()` methods take a permissions string (e.g., 0644, 0777). The static version returns a Folder object. `Jaxer.Dir` expects to be created with *resolved* paths, which means the parameter must be a full path to the file system object. As a contra-example, the following code throws a `NS_ERROR_FILE_UNRECOGNIZED_PATH` error.

```
1. var myFolder = Jaxer.Dir('newfolder');
```

To avoid this issue, use `Jaxer.Dir.resolve(path)` when creating the folder parameter.

```
1. var folderPath = Jaxer.Dir.resolve('newfolder');
2. // Make a Jaxer.Dir object first
3. var myFolder = new Jaxer.Dir(folderPath);
4. myFolder.create('0644');
5. // OR use the static version
6. var myFolder = Jaxer.Dir.create(folderPath, '0644');
```



Getting Information

A folder object has a lot of useful information that you can get using its [accessor methods](#).

The most commonly used method for folder objects is `exists()`. This provides us with a boolean value predicated on the existence of the referenced folder on the file system.

```
1. // if the folder doesn't exist create an empty folder.
2. if (!myFolder.exists())
3. {
4.     myFolder.create();
5. }
```



Now that we have an instantiated folder object called 'myFolder' that we know exists, we can interrogate it to get useful information about the folder and its properties:

```
01. var path = myFolder.path; // C:\aptana\JaxerDev\public\work\new
02. var parentPath = myFolder.parentPath; // C:\aptana\JaxerDev\public\work
03. var dateModified = myFolder.dateModified;
04. // Wed Mar 05 2008 14:52:03 GMT-0800 (Pacific Standard Time)
05.
06. var isDir = myFolder.isDir(); // true
07. var isFile = myFolder.isFile(); // false
08. var isExec = myFolder.isExec(); // false
09. var isSymlink = myFolder.isSymlink(); // false
10. var isWritable = myFolder.isWritable(); // true
11. var isReadable = myFolder.isReadable(); // true
12. var isHidden = myFolder.isHidden(); // false
13. var isSpecial = myFolder.isSpecial(); // false
```



Creating a Folder

Invoke the `create()` method on a dir object.

```
1. var myFolder = new Jaxer.Dir(Jaxer.Dir.resolve('myFolder.txt'));
2. if (!myFolder.exists())
3. {
4.     myFolder.create();
5. }
```



Deleting a Folder

Use the `remove()` method:

```
1. // if the folder exists delete it
2. if (myFolder.exists())
3. {
4.     myFolder.remove();
5. }
```



Navigating the Filesystem

Jaxer includes methods for easily retrieving the contents of folders. However, programmatically navigating the filesystem can result in a lot of scaffolding code so Jaxer also provides `grep` and `map` methods for more powerful scans and operations against the filesystem.

Sample Folder and File Layout

```
01. c:\rootFolder\
02. ----foo.txt
03. ----bar.txt
04. ----subFolder1\
05. -----foo1.txt
```



```

06. -----foo2.txt
07. -----foo3.html (1)
08. -----ignore.html
09. ----subFolder2\
10. -----a.js
11. -----b.js
12. -----c.js
13. ----subFolder3\
14. -----subFolder3a\
15. -----deepF001.txt (2)
16. -----deepF002.txt

```

Example Paths

```

1. (1) c:\rootFolder\subFolder1\foo3.html
2. (2) c:\rootFolder\subFolder3\subFolder3a\deepF001.txt

```

Searching for Files and Folders (Jaxer.Dir's readDir())

Read and process entries in a directory.

```

01. var rootDir = new Jaxer.Dir('c:\\rootFolder');
02. var folderEntries = rootDir.readDir();
03.
04. for (var index = 0; index < folderEntries.length; index++)
05. {
06.     var fileSystemObject = folderEntries[index];
07.
08.     if (fileSystemObject.isFile)
09.     {
10.         // process the file
11.     }
12.     if (fileSystemObject.isDir)
13.     {
14.         // process the file
15.     }
16. }

```

Finding Folders which Match a Pattern (Jaxer.Dir.grep)

Jaxer.Dir.grep takes two parameters, path and options:

- **Path:** directory path to the folder to search.
- **Options:** Javascript Object that contains properties to fine tune the search, including pattern, flags and recursive. **Pattern** is a string containing a JavaScript regular expression, **flags** contains regexp flags to govern the behavior of the pattern parameter and **recursive** indicates whether the search should scan sub-folders.

Simple Grep

```

1. var rootFolder = new Jaxer.Dir("c:\\rootFolder");
2. var myResults = Jaxer.Dir.grep(rootFolder);

```

Simple Grep Results

```

1. c:\rootFolder\bar.txt
2. c:\rootFolder\foo.txt

```

Match a single file in the top-level folder

```

1. var myResults = Jaxer.Dir.grep(rootFolder,{pattern: '^.*foo.*$'}); // c:\rootFolder\foo.txt

```

Recursive Search

```

1. var myResults = Jaxer.Dir.grep(rootFolder,{pattern: '^.*foo.*$',recursive:true});

```

```

1. // returns array containing:
2. c:\rootFolder\foo.txt
3. c:\rootFolder\subFolder1\foo1.txt,
4. c:\rootFolder\subFolder1\foo2.txt
5. c:\rootFolder\subFolder1\foo3.html

```

Make Search Case-Insensitive

```

1. var myResults = Jaxer.Dir.grep(rootFolder,{pattern: '^.*foo.*$',flags:'i',recursive:true});

```

```

1. // returns array of:
2. c:\rootFolder\foo.txt,
3. c:\rootFolder\subFolder1\foo1.txt
4. c:\rootFolder\subFolder1\foo2.txt
5. c:\rootFolder\subFolder1\foo3.html
6. c:\rootFolder\subFolder3\subFolder3a\deepF001.txt
7. c:\rootFolder\subFolder3\subFolder3a\deepF002.txt

```

Object literal Options Parameter

```

1. var opt = {pattern: '^.*[12]\..*$',recursive:true};
2. var myResults = Jaxer.Dir.grep("c:\\rootFolder",opt);

```

```

1. // returns array of:
2. c:\rootFolder\subFolder1\foo1.txt
3. c:\rootFolder\subFolder1\foo2.txt
4. c:\rootFolder\subFolder3\subFolder3a\deepF001.txt
5. c:\rootFolder\subFolder3\subFolder3a\deepF002.txt

```

Operating on Folders which Match a Pattern (Jaxer.Dir.map)

Jaxer.Dir.map adds an optional third parameter to Grep's two, a function to invoke on each matched file object. The function can be inline/anonymous, named reference or left off, in which case the behavior is identical to Grep. The

function gets a single parameter, the File object to operate on.

Since some searches can take a bit of time, depending on the depth of the nesting or complexity of the callback function, take care not to cause a timeout.

Write File Info to a Log

```
1. var myResults = Jaxer.Dir.map(
2.   rootFolder,
3.   {pattern: '^.*[12]\\.*$', recursive: true },
4.   function(f) { Jaxer.Log.info(f.getPath()); return f });
```

```
1. // writes a log entry for each matched file AND
2. // returns the array of Jaxer.File objects:
3. 13:55:19 02/20/2008 [ 2356] [INFO] [JS Framework] [framework.]
   ↵ c:\rootFolder\subFolder1\foo1.txt
4. 13:55:19 02/20/2008 [ 2356] [INFO] [JS Framework] [framework.]
   ↵ c:\rootFolder\subFolder1\foo2.txt
5. 13:55:19 02/20/2008 [ 2356] [INFO] [JS Framework] [framework.]
   ↵ c:\rootFolder\subFolder3\subFolder3a\deepF001.txt
6. 13:55:19 02/20/2008 [ 2356] [INFO] [JS Framework] [framework.]
   ↵ c:\rootFolder\subFolder3\subFolder3a\deepF002.txt
```

Same Code using a Named Callback

```
01. function logMatch(f)
02. {
03.   Jaxer.Log.info(f.getPath());
04.   return f
05. };
06. var opt = {
07.   pattern: '^.*[12]\\.*$',
08.   recursive: true
09. };
10. var myResults = Jaxer.Dir.map(rootFolder, opt, logMatch);
```

Using a Decorator Function to Format the Results

```
01. function fileDecorator(f)
02. {
03.   var decoratorOutput = [];
04.
05.   decoratorOutput.push(f.getLeaf());
06.   decoratorOutput.push(f.getSize());
07.   decoratorOutput.push(f.getExt());
08.   decoratorOutput.push(f.getDateModified());
09.   decoratorOutput.push(f.getParentPath());
10.
11.   return decoratorOutput.join("|");
12. }
```

```
01. var fileInfo = Jaxer.Dir.map(rootFolder, {pattern: '^.*[12]\\.*$', recursive: true
   ↵ fileDecorator});
02. fileInfo.map(function(i) {Jaxer.Log.info(i)});
03.
04. // outputs the following in the log:
05. 15:43:05 02/20/2008 [framework.] foo1.txt|0|txt|Tue
06.   Feb 19 2008 19:15:04 GMT-0800 (Pacific Standard Time)|c:\rootFolder\subFolder1
07. 15:43:05 02/20/2008 [framework.] foo2.txt|0|txt|Tue
08.   Feb 19 2008 19:15:04 GMT-0800 (Pacific Standard Time)|c:\rootFolder\subFolder1
09. 15:43:05 02/20/2008 [framework.] deepF001.txt|0|txt|Tue
10.   Feb 19 2008 19:15:04 GMT-0800 (Pacific Standard
   ↵ Time)|c:\rootFolder\subFolder3\subFolder3a
11. 15:43:05 02/20/2008 [framework.] deepF002.txt|0|txt|Tue
12.   Feb 19 2008 19:15:04 GMT-0800 (Pacific Standard
   ↵ Time)|c:\rootFolder\subFolder3\subFolder3a
```

DirUtils and FileUtils

Jaxer.DirUtils provides access to XPCOM directory/folder utility functions, primarily the following properties:

1. **current:** path to the current folder from which the process is running
2. **desktopDir:** current user's desktop folder
3. **getHome:** current user's home folder
4. **temp:** the Jaxer temp directory

Jaxer.FileUtils provides access to XPCOM filesystem functionality, and is the parent class to both Jaxer.Dir and Jaxer.File.